LLM Engineering Lab 1: Parameter-Efficient Fine-Tuning

1. Introduction: Full Fine-Tuning vs. PEFT

Full fine-tuning updates all weights of a pre-trained foundation model. For large language models, this demands enormous GPU memory because optimizer states (e.g. AdamW storing first and second momentum vectors) require 16 bytes per parameter in FP32, exceeding typical academic workstation capacity. Parameter-Efficient Fine-Tuning (PEFT) freezes the majority of foundation model parameters and trains only a small set of supplementary parameters. This drastically cuts down memory requirements during backpropagation and prevents catastrophic forgetting of broad language capabilities.

In this lab series, our base model is Qwen/Qwen2.5-0.5B-Instruct. We explore Low-Rank Adaptation (LoRA) and Quantized Low-Rank Adaptation (QLoRA) to achieve domain-specific styling without altering underlying factual reasoning.

2. Low-Rank Adaptation (LoRA) Mathematical Principles

LoRA freezes the pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ and constrains its update ΔW with low-rank decomposition: $\Delta W = (\alpha / r) * (B @ A)$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, with rank $r << \min(d, k)$. Matrix A is initialized from a Gaussian distribution $N(0, \sigma^2)$ and matrix B is initialized to zeros, ensuring $\Delta W = 0$ at the start of training. The scalar α is a constant scaling hyperparameter that adjusts the magnitude of adapter updates relative to base features.

Target Modules: In Qwen2.5 models, attention projection layers such as q_proj and v_proj are standard target modules. Setting rank r=4 or r=8 with alpha=16 provides sufficient expressive capacity for stylistic adaptation while tuning less than 0.5% of total model parameters.

3. Hyperparameter Tuning and Overfitting Control

Selecting learning rate and rank requires balancing convergence speed against overfitting. For Qwen2.5-0.5B-Instruct with LoRA, typical learning rates range from 1e-4 to 2e-4 using the AdamW optimizer. A learning rate of 2e-4 with rank r=4 over 2 epochs achieves stable loss reduction without degradation of conversational fluency. Higher learning rates (e.g. 5e-4) or excessive training epochs (>3) risk loss spikes and repetitive syntax generation.

4. Quantized Low-Rank Adaptation (QLoRA) and NF4

QLoRA introduces three core algorithmic innovations to democratize LLM fine-tuning: (1) 4-bit NormalFloat (NF4), an information-theoretically optimal quantile quantization data type for normally distributed weights; (2) Double Quantization (DQ), which quantizes the quantization constants themselves, saving roughly 0.37 bits per parameter; and (3) Paged Optimizers, which leverage CUDA Unified Memory to prevent out-of-memory errors during gradient checkpointing spikes. During forward and backward passes, NF4 weights are dequantized into 16-bit BrainFloat (BF16) or FP16 for computation (bnb_4bit_compute_dtype=torch.float16).

5. Qwen Chat Templates and Delimiter Tokens

Instruction-tuned models like Qwen2.5 require structured chat formatting using ChatML delimiter tokens. Each turn is wrapped with special tokens: <|im_start|>system ... <|im_end|>, <|im_start|>user ... <|im_end|>, and <|im_start|>assistant ... <|im_end|>. The tokenizer.apply_chat_template utility guarantees that tokens are appended identically during training and production inference.